

第3章 动态规划

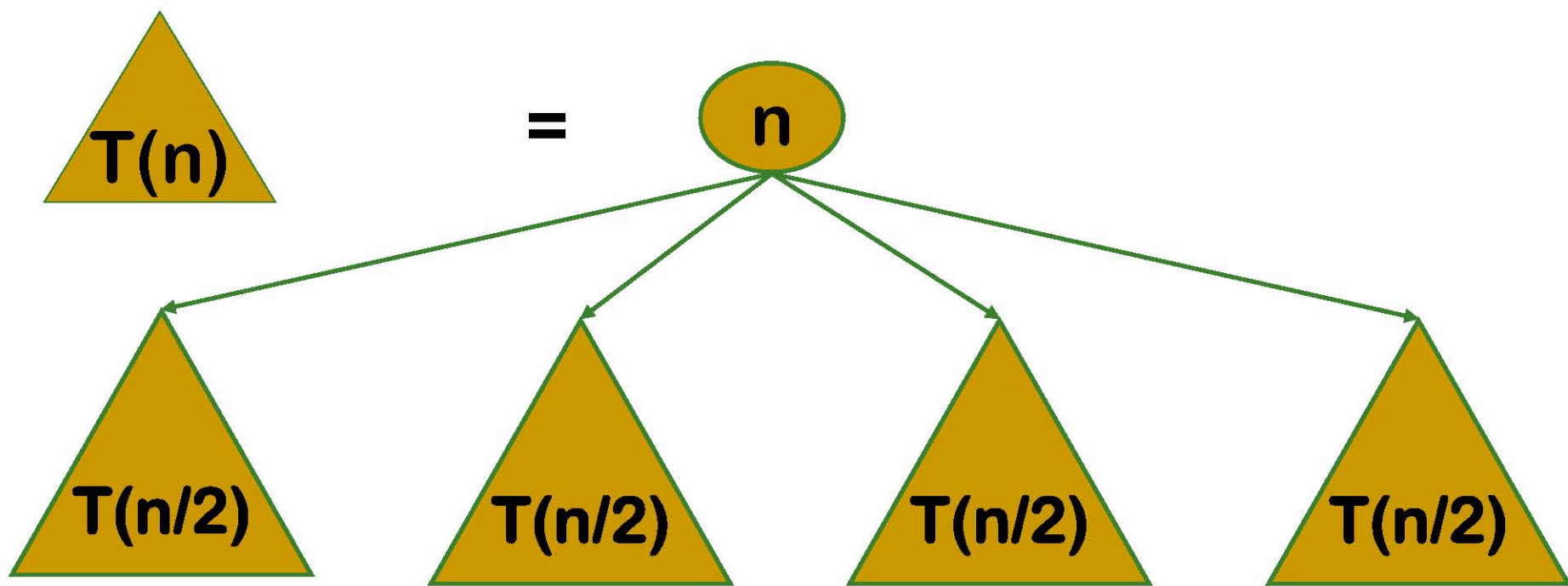
学习要点:

- 理解动态规划算法的概念。
- 掌握动态规划算法的基本要素
 - (1) 最优子结构性质
 - (2) 重叠子问题性质
- 掌握设计动态规划算法的步骤。
 - (1) 找出最优解的性质，并刻画其结构特征。
 - (2) 递归地定义最优值。
 - (3) 以自底向上的方式计算出最优值。
 - (4) 根据计算最优值时得到的信息，构造最优解。

- 通过应用范例学习动态规划算法设计策略。
- （1）矩阵连乘问题；
- （2）最长公共子序列；
- （3）最大子段和
- （4）凸多边形最优三角剖分；
- （5）多边形游戏；
- （6）图像压缩；
- （7）电路布线；
- （8）流水作业调度；
- （9）背包问题；
- （10）最优二叉搜索树。

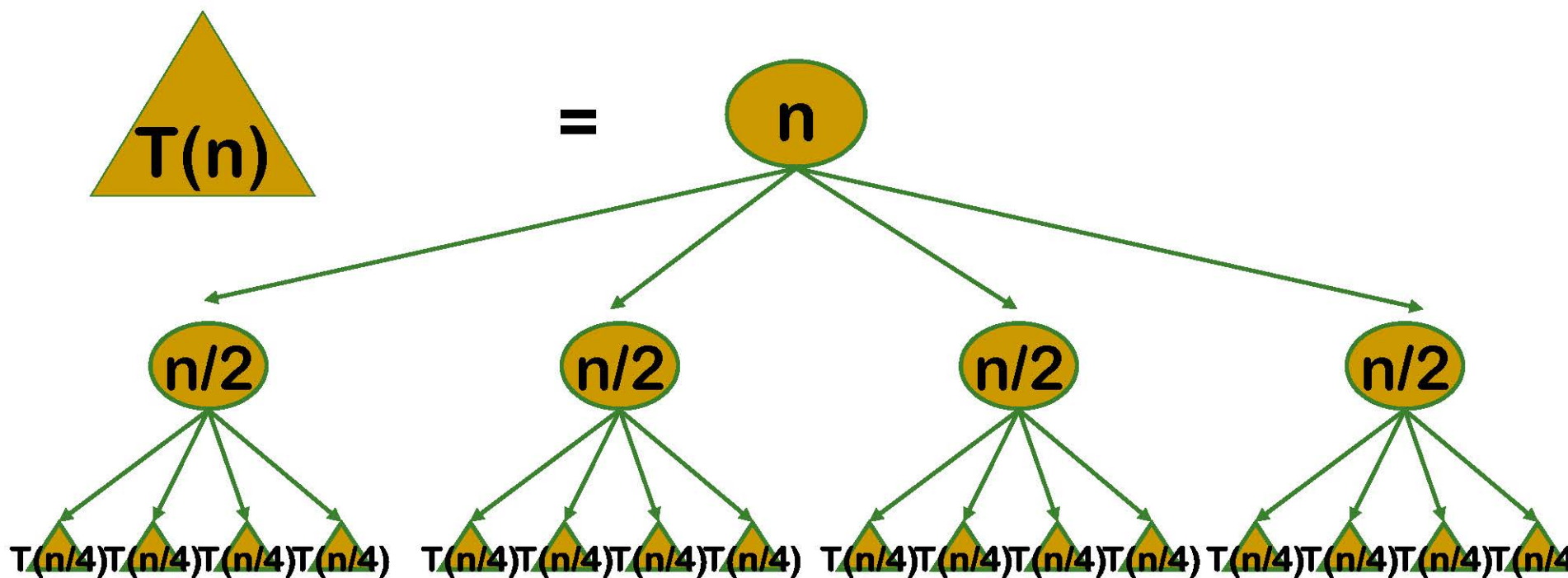
算法总体思想

- 动态规划算法与分治法类似，其基本思想也是将待求解问题分解成若干个子问题



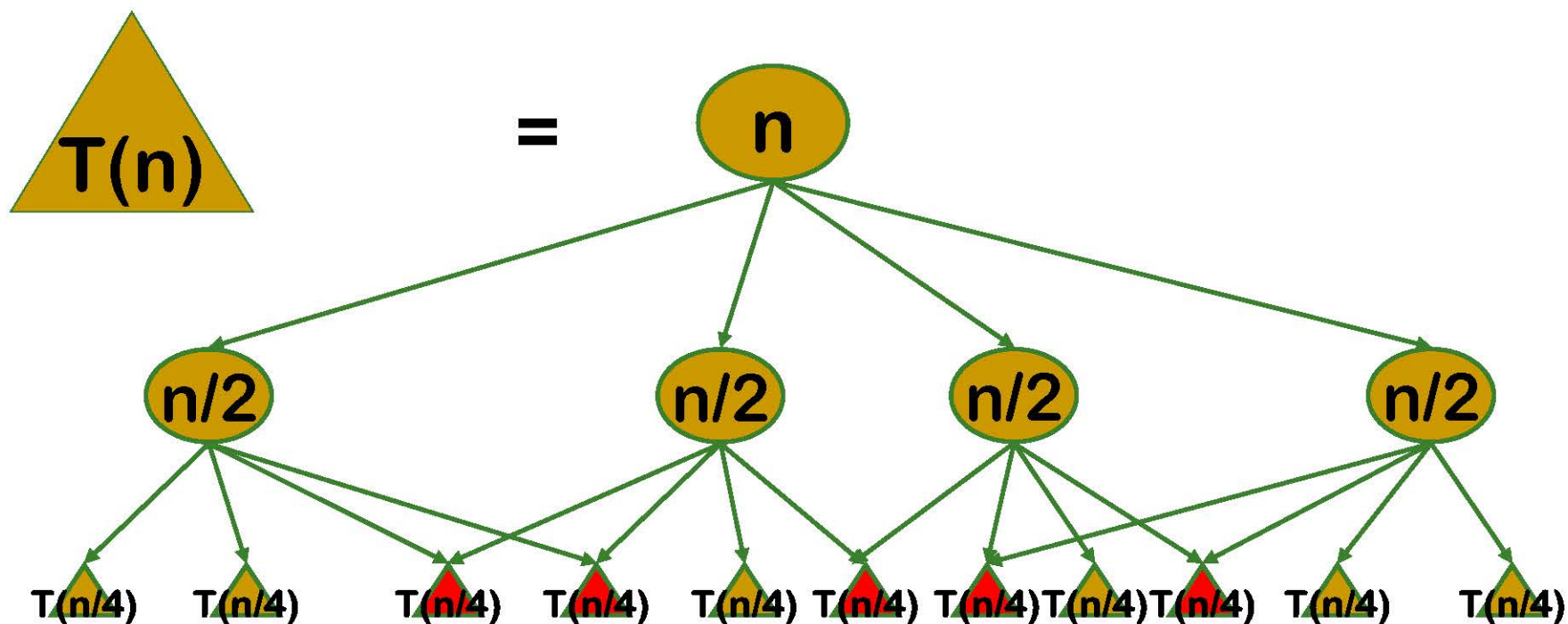
算法总体思想

- 但是经分解得到的子问题往往不是互相独立的。不同子问题的数目常常只有多项式量级。在用分治法求解时，有些子问题被重复计算了许多次。



算法总体思想

- 如果能够保存已解决的子问题的答案，而在需要时再找出已求得的答案，就可以避免大量重复计算，从而得到多项式时间算法。



动态规划基本步骤

- 找出最优解的性质，并刻画其结构特征。
- 递归地定义最优值。
- 以自底向上的方式计算出最优值。
- 根据计算最优值时得到的信息，构造最优解。

完全加括号的矩阵连乘积

- ◆ 完全加括号的矩阵连乘积可递归地定义为：
 - (1) 单个矩阵是完全加括号的；
 - (2) 矩阵连乘积 A 是完全加括号的，则 A 可表示为2个完全加括号的矩阵连乘积 B 和 C 的乘积并加括号，即 $A = (BC)$
- ◆ 设有四个矩阵 A, B, C, D 它们的维数分别是
 $A = 50 \times 10$ $B = 10 \times 40$ $C = 40 \times 30$ $D = 30 \times 5$
- ◆ 总共有五中完全加括号的方式
 $(A((BC)D))$ $(A(B(CD)))$ $((AB)(CD))$
 $((AB)C)D$ $((A(BC))D)$

16000, 10500, 36000, 87500, 34500

矩阵连乘问题

- 给定n个矩阵 $\{A_1, A_2, \dots, A_n\}$ 与 是可乘的,
 $i = 1, 2, \dots$ 考察这n个矩阵的连乘积

$$A_1 A_2 \dots A_n$$

- 由于矩阵乘法满足结合律，所以计算矩阵的连乘可以有許多不同的计算次序。这种计算次序可以用加括号的方式来确定。
- 若一个矩阵连乘积的计算次序完全确定，也就是说该连乘积已完全加括号，则可以依此次序反复调用2个矩阵相乘的标准算法计算出矩阵连乘积

矩阵连乘问题

给定 n 个矩阵 $\{A_1, A_2, \dots, A_n\}$, 其中 A_i 与 A_{i+1} 是可乘的, $i=1, 2, \dots, n-1$ 。如何确定计算矩阵连乘积的计算次序, 使得依此次序计算矩阵连乘积需要的数乘次数最少。

◆**穷举法**: 列举出所有可能的计算次序, 并计算出每一种计算次序相应需要的数乘次数, 从中找出一种数乘次数最少的计算次序。

算法复杂度分析:

对于 n 个矩阵的连乘积, 设其不同的计算次序为 $P(n)$ 。
由于每种加括号方式都可以分解为两个子矩阵的加括号问题:
 $(A_1 \dots A_k)(A_{k+1} \dots A_n)$ 可以得到关于 $P(n)$ 的递推式如下:

$$P(n) = \begin{cases} 1 & n=1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & n>1 \end{cases} \Rightarrow P(n) = \Omega(4^n / n^{3/2})$$

矩阵连乘问题

◆穷举法

◆动态规划

将矩阵连乘积 $A_i A_{i+1} \dots A_j$ 简记为 $A[i:j]$ ，这里 $i \leq j$

考察计算 $A[i:j]$ 的最优计算次序。设这个计算次序在矩阵 A_k 和 A_{k+1} 之间将矩阵链断开， $i \leq k < j$ ，则其相应完全加括号方式为 $(A_i A_{i+1} \dots A_k)(A_{k+1} A_{k+2} \dots A_j)$

计算量： $A[i:k]$ 的计算量加上 $A[k+1:j]$ 的计算量，再加上 $A[i:k]$ 和 $A[k+1:j]$ 相乘的计算量

分析最优解的结构

- 特征：计算 $A[i:j]$ 的最优次序所包含的计算矩阵子链 $A[i:k]$ 和 $A[k+1:j]$ 的次序也是最优的。
- 矩阵连乘计算次序问题的最优解包含着其子问题的最优解。这种性质称为**最优子结构性质**。问题的最优子结构性质是该问题可用动态规划算法求解的显著特征。

建立递归关系

设计算 $A_i * A_{i+1} * \dots * A_j$ 的矩阵连乘积表示为 $A[i:j]$, $1 \leq i \leq j \leq n$, 所需的最少的乘次数为 $m[i,j]$, 则原问题的最优值可表示为 $m[1,n]$

- 当 $i=j$ 时, $A[i:j]=A_i$, 因此, $m[i,i]=0$, $i=1,2,\dots,n$
- 当 $i < j$ 时,

若 $A[i:j]$ 在 A_k 和 A_{k+1} 间断开为最优, 则:

$$A[i:j] = A[i:k] A[k+1:j], \quad i \leq k < j$$

$A[i:k]$ 为 **$A[i]$ 的行数 $\times$ $A[k]$ 的列数** 的矩阵

$A[i]$ 的行 = $A[i-1]$ 的列 = p_{i-1} , $A[k]$ 的列数 = p_k

故: $A[i:k]$ 为 $p_{i-1} * p_k$ 的矩阵

$A[k+1:j]$ 为 $p_k * p_j$ 的矩阵

则所需计算次数: $m[i,j] = m[i,k] + m[k+1,j] + p_{i-1}p_kp_j$

建立递归关系

对 $A[i:j]$ 来说, 当 $A[i:j] = A[i:k] A[k+1:j]$, $i \leq k < j$,

$$m[i, j] = m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j$$

K取值有 $j-i$ 种, 因此递归地定义 $m[i, j]$ 为:

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k + 1, j] + p_{i-1}p_kp_j\} & i < j \end{cases}$$

计算最优值

- 对于 $1 \leq i \leq j \leq n$ 不同的有序对 (i, j) 对应于不同的子问题。因此，不同子问题的个数最多只有

$$\binom{n}{2} + n = \Theta(n^2)$$

- 由此可见，在递归计算时，**许多子问题被重复计算多次**。这也是该问题可用动态规划算法求解的又一显著特征。
- 用动态规划算法解此问题，可依据其递归式以自底向上的方式进行计算。在计算过程中，保存已解决的子问题答案。每个子问题只计算一次，而在后面需要时只要简单查一下，从而避免大量的重复计算，最终得到多项式时间的算法

用动态规划法求最优解

```
void MatrixChain(int *p, int n, int **m, int **s)
```

```
{
```

```
    for (int i = 1; i <= n; i++) m[i][i] = 0;
```

```
    for (int r = 2; r <= n; r++) /* r 为连乘矩阵的数量 */
```

```
        for (int i = 1; i <= n - r + 1; i++) {
```

```
            int j = i + r - 1;
```

```
            m[i][j] = m[i+1][j] + p[i-1]*p[i]*p[j];
```

```
            s[i][j] = i;
```

```
            for (int k = i+1; k < j; k++) {
```

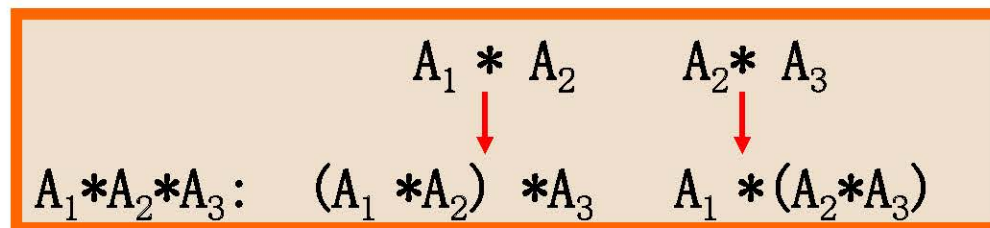
```
                int t = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
```

```
                if (t < m[i][j]) { m[i][j] = t; s[i][j] = k; }
```

```
            }
```

```
        }
```

```
}
```



算法思路:

先计算2个矩阵连乘

再计算3个矩阵连乘, (利用2个矩阵的连乘结果, 避免重复计算)

。 。 。

计算n个矩阵连乘

用动态规划法求最优解

```
void MatrixChain(int *p, int n, int **m, int **s)
```

```
{  
    for (int i = 1; i <= n; i++) m[i][i] = 0;  
    for (int r = 2; r <= n; r++)  
        for (int i = 1; i <= n - r + 1; i++) {  
            int j = i + r - 1;  
            m[i][j] = m[i + 1][j] + p[i - 1] * p[i] * p[j];  
            s[i][j] = i;  
            for (int k = i + 1; k < j; k++) {  
                int t = m[i][k] + m[k + 1][j] + p[i - 1] * p[k] * p[j];  
                if (t < m[i][j]) { m[i][j] = t; s[i][j] = k; }  
            }  
        }  
}
```

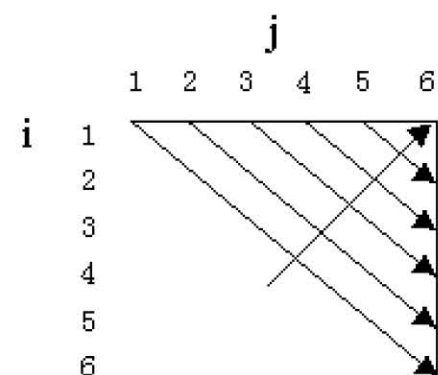
算法复杂度分析：

MatrixChain的主要计算量取决于算法中对 r , i 和 k 的3重循环。循环体内的计算量为 $O(1)$ ，而3重循环的总次数为 $O(n^3)$ 。因此算法的计算时间上界为 $O(n^3)$ 。算法所占用的空间显然为 $O(n^2)$ 。

用动态规划法求最优解

A1	A2	A3	A4	A5	A6
30×35	35×15	15×5	5×10	10×20	20×25

$$m[2][5] = \min \begin{cases} m[2][2] + m[3][5] + p_1 p_2 p_5 = 0 + 2500 + 35 \times 15 \times 20 = 13000 \\ m[2][3] + m[4][5] + p_1 p_3 p_5 = 2625 + 1000 + 35 \times 5 \times 20 = 7125 \\ m[2][4] + m[5][5] + p_1 p_4 p_5 = 4375 + 0 + 35 \times 10 \times 20 = 11375 \end{cases}$$



(a) 计算次序

	j	1	2	3	4	5	6
i	1	0	15750	7875	9375	11875	15125
	2		0	2625	4375	7125	10500
	3			0	750	2500	5375
	4				0	1000	3500
	5					0	5000
	6						0

(b) $m[i][j]$

	j	1	2	3	4	5	6
i	1	0	1	1	3	3	3
	2		0	2	3	3	3
	3			0	3	3	3
	4				0	4	5
	5					0	5
	6						0

(c) $s[i][j]$

用动态规划法构造最优解

$s[i][j]$ 的值为计算 $A[i:j]$ 的最小乘次数的断开位置。

```
void Traceback ( int i, int j, int **s )
{
    if ( i == j )
        return;

    Traceback ( i, s[i][j], s);
    Traceback ( s[i][j] + 1, j, s);

    cout << "Multiply A" << i << "," << s[ i, j ] ;
    cout << " and A " << ( s[ i, j ] + 1 ) << ", " << j << endl;
}
```

动态规划算法的基本要素

一、最优子结构

- 矩阵连乘计算次序问题的最优解包含着其子问题的最优解。这种性质称为**最优子结构性质**。
- 在分析问题的最优子结构性质时，所用的方法具有普遍性：首先假设由问题的最优解导出的子问题的解不是最优的，然后再设法说明在这个假设下可构造出比原问题最优解更好的解，从而导致矛盾。
- 利用问题的最优子结构性质，以自底向上的方式递归地从子问题的最优解逐步构造出整个问题的最优解。最优子结构是问题能用动态规划算法求解的前提。

同一个问题可以有多种方式刻画它的最优子结构，有些表示方法的求解速度更快（空间占用小，问题的维度低）

动态规划算法的基本要素

二、重叠子问题

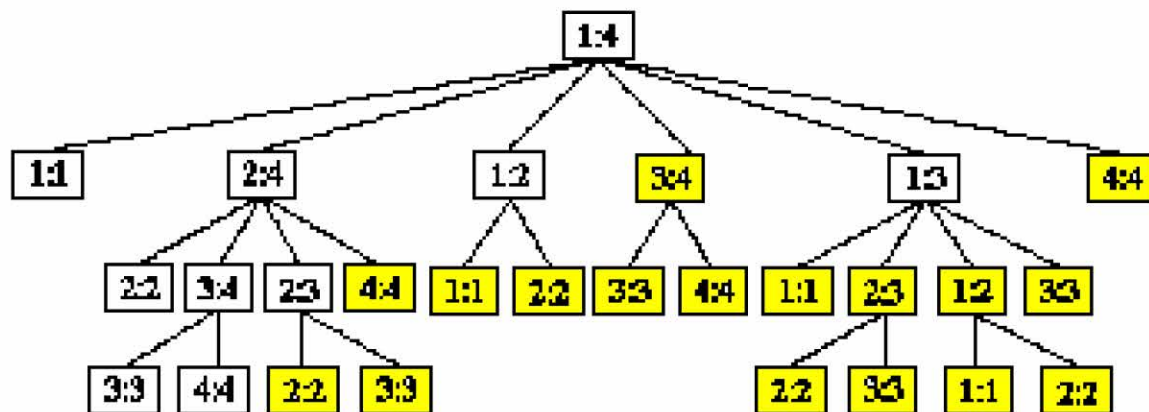
```
void RecurMatrixChain(int i, int j)
{
    if ( i == j )
        return 0;

    int u = RecurMatrixChain(i, i) + RecurMatrixChain(i+1, j)+ p[i-1]*p[i]*p[j];
    s[i][j] = i;
    for (int k=i+1; k < j; k++) {
        int t = RecurMatrixChain(i, k) + RecurMatrixChain(k+1, j)+ p[i-1]*p[k]*p[j];
        if (t < u) { u = t; s[i][j] = k; }
    }
}
```

动态规划算法的基本要素

二、重叠子问题

- 递归算法求解问题时，每次产生的子问题并不总是新问题，有些子问题被反复计算多次。这种性质称为子问题的**重叠性质**。
- 动态规划算法，对每一个子问题只解一次，而后将其解保存在一个表格中，当再次需要解此子问题时，只是简单地用常数时间查看一下结果。
- 通常不同的子问题个数随问题的大小呈多项式增长。因此用动态规划算法只需要多项式时间，从而获得较高的解题效率。



动态规划算法的基本要素

二、重叠子问题

$$T(n) \geq \begin{cases} O(1) & n = 1 \\ 1 + \sum_{k=1}^{n-1} (T(k) + T(n-k)) + 1 & n > 1 \end{cases}$$

$$T(n) \geq 2^{n-1} = \Omega(2^n)$$

动态规划算法的基本要素

三、备忘录方法

- 备忘录方法的控制结构与直接递归方法的控制结构相同，区别在于备忘录方法为每个解过的子问题建立了备忘录以备需要时查看，避免了相同子问题的重复求解。

```
int LookupChain(int i, int j)
{
    if (m[i][j] > 0) return m[i][j];
    if (i == j) return 0;
    int u = LookupChain(i, i) + LookupChain(i+1, j) + p[i-1]*p[i]*p[j];
    s[i][j] = i;
    for (int k = i+1; k < j; k++) {
        int t = LookupChain(i, k) + LookupChain(k+1, j) + p[i-1]*p[k]*p[j];
        if (t < u) { u = t; s[i][j] = k;}
    }
    m[i][j] = u;
    return u;
}
```

最长公共子序列

给定序列 $X = \{x_1, x_2, \dots, x_m\}$ ，另一序列 $Z = \{z_1, z_2, \dots, z_m\}$ 。
若 Z 是 X 的子序列，是指存在一个严格递增下标序列
 $\{i_1, i_2, \dots, i_k\}$ ，使得对于所有 $j=1, 2, \dots, k$ 有： $z_j = x_{i_j}$ 。

例如：序列 $Z = \{B, C, D, B\}$ 是序列 $X = \{A, B, C, B, D, A, B\}$ 的子序列，相应的递增下标序列为 $\{2, 3, 5, 7\}$ 。

给定2个序列 X 和 Y ，当另一序列 Z 既是 X 的子序列又是 Y 的子序列时，称 Z 是序列 X 和 Y 的公共子序列。

给定2个序列 $X = \{x_1, x_2, \dots, x_m\}$ 和 $Y = \{y_1, y_2, \dots, y_n\}$ ，找出 X 和 Y 的最长公共子序列。

最长公共子序列的结构

设序列 $X=\{x_1, x_2, \dots, x_m\}$ 和 $Y=\{y_1, y_2, \dots, y_n\}$ 的最长公共子序列为 $Z=\{z_1, z_2, \dots, z_k\}$ ，则

- (1) 若 $x_m = y_n$ ，则 $z_k = x_m = y_n$ ，且 Z_{k-1} 是 X_{m-1} 和 Y_{n-1} 的最长公共子序列。
- (2) 若 $x_m \neq y_n$ 且 $z_k \neq x_m$ ，则 Z 是 X_{m-1} 和 Y 的最长公共子序列。
- (3) 若 $x_m \neq y_n$ 且 $z_k \neq y_n$ ，则 Z 是 X 和 Y_{n-1} 的最长公共子序列。

其中，

$X_{m-1} = \{x_1, x_2, \dots, x_{m-1}\}$, $Y_{n-1} = \{y_1, y_2, \dots, y_{n-1}\}$, $Z_{k-1} = \{z_1, z_2, \dots, z_{k-1}\}$

由此可见，2个序列的最长公共子序列包含了这2个序列的前缀的最长公共子序列。

最长公共子序列问题具有最优子结构性质。

子问题的递归结构

由最长公共子序列问题的最优子结构性性质建立子问题最优值的递归关系。用 $c[i][j]$ 记录序列 X 和 Y 的最长公共子序列的长度。其中, $X_i = \{x_1, x_2, \dots, x_i\}$; $Y_j = \{y_1, y_2, \dots, y_j\}$ 。当 $i=0$ 或 $j=0$ 时, 空序列是 X_i 和 Y_j 的最长公共子序列。故此时 $C[i][j]=0$ 。其它情况下, 由最优子结构性性质可建立递归关系如下:

$$c[i][j] = \begin{cases} 0 & i = 0, j = 0 \\ c[i-1][j-1] + 1 & i, j > 0; x_i = y_j \\ \max\{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_j \end{cases}$$

计算最优值

由于在所考虑的子问题空间中，总共有 $\theta(mn)$ 个不同的子问题，因此，用动态规划算法自底向上地计算最优值能提高算法的效率。

```
void LCSLength(int m, int n, char *x,
char *y, int **c, int **b)
{
    int i, j;
    for (i = 1; i <= m; i++) c[i][0] = 0;
    for (i = 1; i <= n; i++) c[0][i] = 0;
    for (i = 1; i <= m; i++)
        for (j = 1; j <= n; j++) {
            if (x[i]==y[j]) {
                c[i][j]=c[i-1][j-1]+1; b[i][j]=1;}
            else if (c[i-1][j]>=c[i][j-1]) {
                c[i][j]=c[i-1][j]; b[i][j]=2;}
            else { c[i][j]=c[i][j-1]; b[i][j]=3; }
        }
}
```

构造最长公共子序列

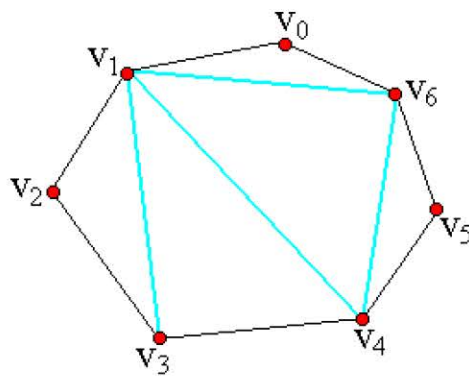
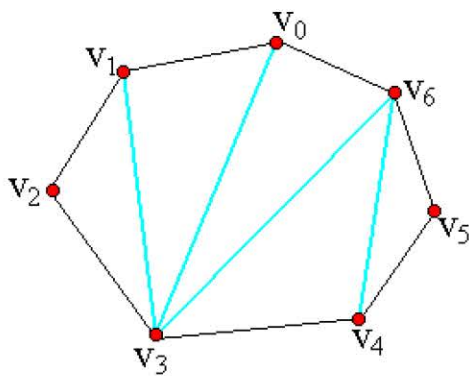
```
void LCS(int i, int j, char *x, int **b)
{
    if (i ==0 || j==0) return;
    if (b[i][j]== 1){ LCS(i-1, j-1, x, b); cout<<x[i]; }
    else if (b[i][j]== 2) LCS(i-1, j, x, b);
    else LCS(i, j-1, x, b);
}
```

算法的改进

- 在算法 `lcsLength` 和 `lcs` 中，可进一步将数组 `b` 省去。事实上，数组元素 `c[i][j]` 的值仅由 `c[i-1][j-1]`，`c[i-1][j]` 和 `c[i][j-1]` 这3个数组元素的值所确定。对于给定的数组元素 `c[i][j]`，可以不借助于数组 `b` 而仅借助于 `c` 本身在时间内确定 `c[i][j]` 的值是由 `c[i-1][j-1]`，`c[i-1][j]` 和 `c[i][j-1]` 中哪一个值所确定的。
- 如果只需要计算最长公共子序列的长度，则算法的空间需求可大大减少。事实上，在计算 `c[i][j]` 时，只用到数组 `c` 的第 `i` 行和第 `i-1` 行。因此，用2行的数组空间就可以计算出最长公共子序列的长度。进一步的分析还可将空间需求减至 $O(\min(m,n))$ 。

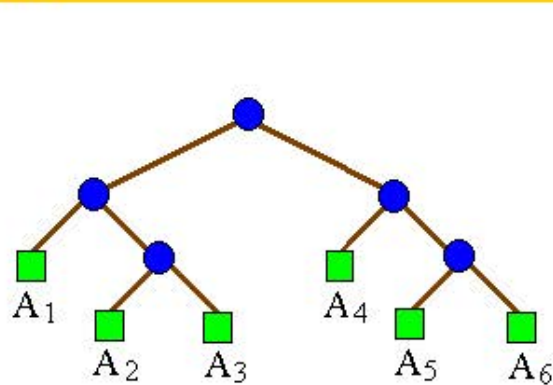
凸多边形最优三角剖分

- 用多边形顶点的逆时针序列表示凸多边形，即 $P=\{v_0, v_1, \dots, v_{n-1}\}$ 表示具有 n 条边的凸多边形。
- 若 v_i 与 v_j 是多边形上不相邻的2个顶点，则线段 $v_i v_j$ 称为多边形的一条弦。弦将多边形分割成2个多边形 $\{v_i, v_{i+1}, \dots, v_j\}$ 和 $\{v_j, v_{j+1}, \dots, v_i\}$ 。
- 多边形的三角剖分是将多边形分割成互不相交的三角形的弦的集合 T 。
- 给定凸多边形 P ，以及定义在由多边形的边和弦组成的三角形上的权函数 w 。要求确定该凸多边形的三角剖分，使得即该三角剖分中诸三角形上权之和为最小。

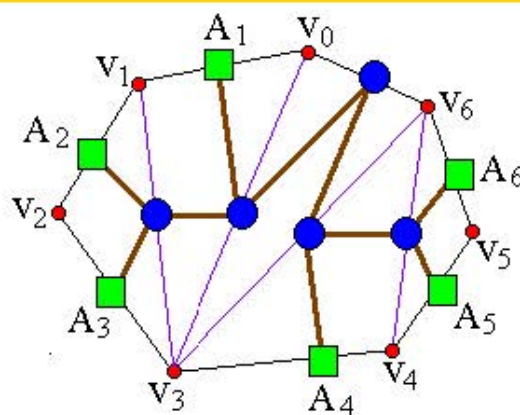


三角剖分的结构及其相关问题

- 一个表达式的完全加括号方式相应于一棵完全二叉树，称为表达式的语法树。例如，完全加括号的矩阵连乘积 $((A_1(A_2A_3))(A_4(A_5A_6)))$ 所相应的语法树如图 (a) 所示。
- 凸多边形 $\{v_0, v_1, \dots, v_{n-1}\}$ 的三角剖分也可以用语法树表示。例如，图 (b) 中凸多边形的三角剖分可用图 (a) 所示的语法树表示。
- 矩阵连乘积中的每个矩阵 A_i 对应于凸 $(n+1)$ 边形中的一条边 $v_{i-1}v_i$ 。三角剖分中的一条弦 v_iv_j , $i < j$, 对应于矩阵连乘积 $A[i+1:j]$ 。



(a)



(b)

最优子结构性质

- 凸多边形的最优三角剖分问题有最优子结构性质。
- 事实上，若凸 $(n+1)$ 边形 $P=\{v_0, v_1, \dots, v_n\}$ 的最优三角剖分 T 包含三角形 $v_0 v_k v_n$, $1 \leq k \leq n-1$, 则 T 的权为3个部分权的和：三角形 $v_0 v_k v_n$ 的权，子多边形 $\{v_0, v_1, \dots, v_k\}$ 和 $\{v_k, v_{k+1}, \dots, v_n\}$ 的权之和。可以断言，由 T 所确定的这2个子多边形的三角剖分也是最优的。因为若有 $\{v_0, v_1, \dots, v_k\}$ 或 $\{v_k, v_{k+1}, \dots, v_n\}$ 的更小权的三角剖分将导致 T 不是最优三角剖分的矛盾。

最优三角剖分的递归结构

• 定义 $t[i][j]$, $1 \leq i < j \leq n$ 为凸子多边形 $\{v_{i-1}, v_i, \dots, v_j\}$ 的最优三角剖分所对应的权函数值, 即其最优值。为方便起见, 设退化的多边形 $\{v_{i-1}, v_i\}$ 具有权值 0。据此定义, 要计算的凸 $(n+1)$ 边形 P 的最优权值为 $t[1][n]$ 。

• $t[i][j]$ 的值可以利用最优子结构性性质递归地计算。当 $j-i \geq 1$ 时, 凸子多边形至少有 3 个顶点。由最优子结构性性质, $t[i][j]$ 的值应为 $t[i][k]$ 的值加上 $t[k+1][j]$ 的值, 再加上三角形 $v_{i-1}v_kv_j$ 的权值, 其中 $i \leq k \leq j-1$ 。由于在计算时还不知道 k 的确切位置, 而 k 的所有可能位置只有 $j-i$ 个, 因此可以在这 $j-i$ 个位置中选出使 $t[i][j]$ 值达到最小的位置。由此, $t[i][j]$ 可递归地定义为:

$$t[i][j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{t[i][k] + t[k+1][j] + w(v_{i-1}v_kv_j)\} & i < j \end{cases}$$

最优三角剖分

```
void MinWeightTriangulation(int n, Type **t, int **s)
{
    for (int i = 1; i <= n; i++) t[i][i] = 0;
    for (int r = 2; r <= n; r++) /* 多边形节点个数为r+1 */
        for (int i = 1; i <= n - r + 1; i++) {
            int j = i + r - 1;
            t[i][j] = t[i+1][j] + w(i-1, i, j);
            s[i][j] = i;
            for (int k = i+1; k < i+r-1; k++) { /* j = i+r-1 */
                int u = t[i][k] + t[k+1][j] + w(i-1, k, j);
                if (u < t[i][j]) {
                    t[i][j] = u;
                    s[i][j] = k;
                }
            }
        }
    }
}
```

多边形游戏

多边形游戏是一个单人玩的游戏，开始时有一个由 n 个顶点构成的多边形。每个顶点被赋予一个整数值，每条边被赋予一个运算符“+”或“*”。所有边依次用整数从1到 n 编号。

游戏第1步，将一条边删除。

随后 $n-1$ 步按以下方式操作：

- (1) 选择一条边 E 以及由 E 连接着的2个顶点 $V1$ 和 $V2$ ；
- (2) 用一个新的顶点取代边 E 以及由 E 连接着的2个顶点 $V1$ 和 $V2$ 。将由顶点 $V1$ 和 $V2$ 的整数值通过边 E 上的运算得到的结果赋予新顶点。

最后，所有边都被删除，游戏结束。游戏的得分就是所剩顶点上的整数值。

问题：对于给定的多边形，计算最高得分。

最优子结构性质

- 在所给多边形中，从顶点 i ($1 \leq i \leq n$) 开始，长度为 j (链中有 j 个顶点) 的顺时针链 $p(i, j)$ 可表示为 $v[i], op[i+1], \dots, v[i+j-1]$ 。
 - 如果这条链的最后一次合并运算在 $op[i+s]$ 处发生 ($1 \leq s \leq j-1$)，则可在 $op[i+s]$ 处将链分割为2个子链 $p(i, s)$ 和 $p(i+s, j-s)$ 。
 - 设 m_1 是对子链 $p(i, s)$ 的任意一种合并方式得到的值，而 a 和 b 分别是在所有可能的合并中得到的最小值和最大值。 m_2 是 $p(i+s, j-s)$ 的任意一种合并方式得到的值，而 c 和 d 分别是在所有可能的合并中得到的最小值和最大值。依此定义有 $a \leq m_1 \leq b$, $c \leq m_2 \leq d$
- (1) 当 $op[i+s]='+'$ 时，显然有 $a+c \leq m \leq b+d$
- (2) 当 $op[i+s]='*'$ 时，有 $\min\{ac, ad, bc, bd\} \leq m \leq \max\{ac, ad, bc, bd\}$
- 换句话说，主链的最大值和最小值可由子链的最大值和最小值得到。

图像压缩

图象的变位压缩存储格式将所给的像素点序列 $\{p_1, p_2, \dots, p_n\}$, $0 \leq p_i \leq 255$ 分割成 m 个连续段 $S_1, S_2, \dots, S_m$ 。第 i 个像素段 S_i 中 ($1 \leq i \leq m$)，有 $l[i]$ 个像素，且该段中每个像素都只用 $b[i]$ 位表示。设 $t[i] = \sum_{k=1}^i l[k]$ 则第 i 个像素段 S_i 为 $S_i = \{p_{t[i]+1}, \dots, p_{t[i]+l[i]}\}$

设 $h_i = \left\lceil \log \left(\max_{t[i]+1 \leq k \leq t[i]+l[i]} p_k + 1 \right) \right\rceil$ ，则 $h_i \leq b[i] \leq 8$ 。因此需要用 3 位表示 $b[i]$ ，如果限制 $1 \leq l[i] \leq 255$ ，则需要用 8 位表示 $l[i]$ 。因此，第 i 个像素段所需的存储空间为 $l[i] * b[i] + 11$ 位。按此格式存储像素序列 $\{p_1, p_2, \dots, p_n\}$ ，需要 $\sum_{i=1}^m l[i] * b[i] + 11m$ 位的存储空间。

图象压缩问题要求确定像素序列 $\{p_1, p_2, \dots, p_n\}$ 的最优分段，使得依此分段所需的存储空间最少。每个分段的长度不超过 256 位。

图像压缩

设 $l[i]$, $b[i]$, 是 $\{p_1, p_2, \dots, p_n\}$ 的最优分段。显而易见, $l[1]$, $b[1]$ 是 $\{p_1, \dots, p_{l[1]}\}$ 的最优分段, 且 $l[i]$, $b[i]$, 是 $\{p_{l[1]+1}, \dots, p_n\}$ 的最优分段。即图像压缩问题满足最优子结构性质。

设 $s[i]$, $1 \leq i \leq n$, 是像素序列 $\{p_1, \dots, p_n\}$ 的最优分段所需的存储位数。由最优子结构性质易知:

$$s[i] = \min_{1 \leq k \leq \min\{i, 256\}} \{s[i-k] + k * b_{\max(i-k+1, i)}\} + 1$$

其中 $b_{\max(i, j)} = \left\lceil \log \left(\max_{i \leq k \leq j} \{p_k\} + 1 \right) \right\rceil$

算法复杂度分析:

由于算法 **compress** 中对 k 的循环次数不超过 256, 故对每一个确定的 i , 可在时间 $O(1)$ 内完成的计算。因此整个算法所需的计算时间为 $O(n)$ 。

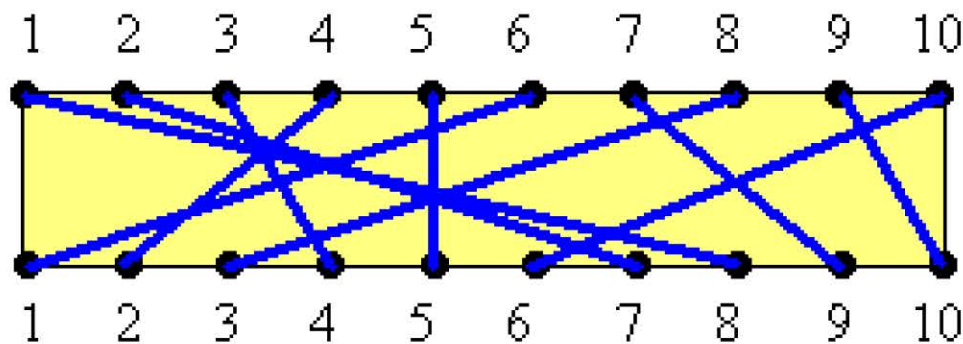
void **Compress**(int n, int p[], int s[], int l[], int b[]) //s[i] 第i个像素段长度

```
{  
    int Lmax = 256, header = 11;  
    s[0] = 0;  
    for (int i = 1; i <= n; i++) {    // 像素点  
        b[i] = length(p[i]);          // 像素点灰度值所占bit位  
        int bmax = b[i];  
        s[i] = s[i-1] + bmax;  
        l[i] = 1;  
        for (int j = 2; j <= i && j <= Lmax; j++) { //像素段长度, i为该段最后一个节点  
            if (bmax <= b[i-j+1]) bmax = b[i-j+1];  
            if( s[i] > s[i-j] + j * bmax ) {  
                s[i] = s[i-j] + j * bmax;  
                l[i] = j;  
            }  
        }  
        s[i] += header;  
    }  
}
```

电路布线

在一块电路板的上、下2端分别有 n 个接线柱。根据电路设计，要求用导线 $(i, \pi(i))$ 将上端接线柱与下端接线柱相连，如图所示。其中 $\pi(i)$ 是 $\{1, 2, \dots, n\}$ 的一个排列。导线 $(i, \pi(i))$ 称为该电路板上的第 i 条连线。对于任何 $1 \leq i < j \leq n$ ，第 i 条连线和第 j 条连线相交的充分且必要的条件是 $\pi(i) > \pi(j)$ 。

电路布线问题要确定将哪些连线安排在第一层上，使得该层上有尽可能多的连线。换句话说，该问题要求确定导线集 $\text{Nets} = \{(i, \pi(i)), 1 \leq i \leq n\}$ 的最大不相交子集。



电路布线

记 $N(i, j) = \{t \mid (t, \pi(t)) \in Nets, t \leq i, \pi(t) \leq j\}$ 。N(i,j)的最大不相交子集为MNS(i,j)。Size(i,j)=|MNS(i,j)|。

(1)当i=1时, $MNS(1, j) = N(1, j) = \begin{cases} \emptyset & j < \pi(1) \\ \{(1, \pi(1))\} & j \geq \pi(1) \end{cases}$

(2)当i>1时,

2.1 $j < \pi(i)$ 此时 $(i, \pi(i)) \notin N(i, j)$ 故在这种情况下

(1)当i=1时 $Size(1, j) = \begin{cases} 0 & j < \pi(1) \\ 1 & j \geq \pi(1) \end{cases}$

2.2 $j \geq \pi(i)$

则对 (2)当i>1时

若 $Size(i, j) = \begin{cases} Size(i-1, j) & j < \pi(i) \\ \max\{Size(i-1, j), Size(i-1, \pi(i)-1) + 1\} & j \geq \pi(i) \end{cases}$

$t < i$ 。从而 $MNS(i, j) \subseteq N(i-1, j)$ 因此, $Size(i, j) \leq Size(i-1, j)$ 。
另一方面 $MNS(i-1, j) \subseteq N(i, j)$, 故又有 $Size(i, j) \geq Size(i-1, j)$,
从而 $Size(i, j) = Size(i-1, j)$ 。

电路布线最优值

```
void MNS(int C[ ], int n, int **size)
{
    for (j = 0; j < C[1]; j++)
        size[1][j] = 0;

    for (j = C[1]; j < C[1]; j++)
        size[1][j] = 1;

    for (i = 2; i < n; i++) {

        for( int j = 0; j < c[i]; j++)
            size [i][j] = size[i-1][j];

        for( int j = c[i]; j <= n; j++)
            size[i][j] = max(size[i-1][j], size[i-1][C[i]-1]+1);
    }
    size[n][n] = max(size([n-1][n], size[n-1][C[n]-1] + 1);
}
```

流水作业调度

n 个作业 $\{1, 2, \dots, n\}$ 要在由2台机器 $M1$ 和 $M2$ 组成的流水线上完成加工。每个作业加工的顺序都是先在 $M1$ 上加工，然后在 $M2$ 上加工。 $M1$ 和 $M2$ 加工作业 i 所需的时间分别为 a_i 和 b_i 。

流水作业调度问题要求确定这 n 个作业的最优加工顺序，使得从第一个作业在机器 $M1$ 上开始加工，到最后一个作业在机器 $M2$ 上加工完成所需的时间最少。

分析：

- 直观上，一个最优调度应使机器 $M1$ 没有空闲时间，且机器 $M2$ 的空闲时间最少。在一般情况下，机器 $M2$ 上会有机器空闲和作业积压2种情况。
- 设全部作业的集合为 $N=\{1, 2, \dots, n\}$ 。 $S \subseteq N$ 是 N 的作业子集。在一般情况下，机器 $M1$ 开始加工 S 中作业时，机器 $M2$ 还在加工其它作业，要等时间 t 后才可利用。将这种情况下完成 S 中作业所需的最短时间记为 $T(S, t)$ 。流水作业调度问题的最优值为 $T(N, 0)$ 。

流水作业调度

设 π 是所给 n 个流水作业的一个最优调度，它所需的加工时间为 $a_{\pi(1)} + T'$ 。其中 T' 是在机器 M_2 的等待时间为 $b_{\pi(1)}$ 时，安排作业 $\pi(2), \dots, \pi(n)$ 所需的时间。

记 $S = N - \{\pi(1)\}$ ，则有 $T' = T(S, b_{\pi(1)})$ 。

证明：事实上，由 T 的定义知 $T' \geq T(S, b_{\pi(1)})$ 。若 $T' > T(S, b_{\pi(1)})$ ，设 π' 是作业集 S 在机器 M_2 的等待时间为 $b_{\pi(1)}$ 情况下的一个最优调度。则 $\pi(1), \pi'(2), \dots, \pi'(n)$ 是 N 的一个调度，且该调度所需的时间为 $a_{\pi(1)} + T(S, b_{\pi(1)}) < a_{\pi(1)} + T'$ 。这与 π 是 N 的最优调度矛盾。故 $T' \leq T(S, b_{\pi(1)})$ 。从而 $T' = T(S, b_{\pi(1)})$ 。这就证明了流水作业调度问题具有最优子结构的性质。

由流水作业调度问题的最优子结构性性质可知，

$$T(N, 0) = \min_{1 \leq i \leq n} \{a_i + T(N - \{i\}, b_i)\}$$

$$T(S, t) = \min_{i \in S} \{a_i + T(S - \{i\}, b_i + \max\{t - a_i, 0\})\}$$

Johnson不等式

对递归式的深入分析表明，算法可进一步得到简化。

设 π 是作业集 S 在机器 M_2 的等待时间为 t 时的任一最优调度。若 $\pi(1)=i, \pi(2)=j$ 。则由动态规划递归式可得：

$$T(S, t) = a_i + T(S - \{i\}, b_i + \max\{t - a_i, 0\}) = a_i + a_j + T(S - \{i, j\}, t_{ij})$$

$$\begin{aligned} \text{其中, } t_{ij} &= b_j + \max\{b_i + \max\{t - a_i, 0\} - a_j, 0\} \\ &= b_j + b_i - a_j + \max\{\max\{t - a_i, 0\}, a_j - b_i\} \\ &= b_j + b_i - a_j + \max\{t - a_i, a_j - b_i, 0\} \\ &= b_j + b_i - a_j - a_i + \max\{t, a_i + a_j - b_i, a_i\} \end{aligned}$$

如果作业 i 和 j 满足 $\min\{b_i, a_j\} \geq \min\{b_j, a_i\}$ ，则称作业 i 和 j 满足**Johnson不等式**。

流水作业调度的Johnson法则

$$t_{ij} = b_j + b_i - a_j - a_i + \max\{t, a_i + a_j - b_i, a_i\}$$

交换作业i和作业j的加工顺序, 得到作业集S的另一调度, 它所需的加工时间为 $T'(S, t) = a_i + a_j + T(S - \{i, j\}, t_{ji})$

$$\text{其中 } t_{ji} = b_j + b_i - a_j - a_i + \max\{t, a_i + a_j - b_j, a_j\}$$

当作业i和j满足Johnson不等式时, 有

$$\max\{-b_i, -a_j\} \leq \max\{-b_j, -a_i\}$$

$$a_i + a_j + \max\{-b_i, -a_j\} \leq a_i + a_j + \max\{-b_j, -a_i\}$$

$$\max\{a_i + a_j - b_i, a_i\} \leq \max\{a_i + a_j - b_j, a_j\}$$

$$\max\{t, a_i + a_j - b_i, a_i\} \leq \max\{t, a_i + a_j - b_j, a_j\}$$

由此可见当作业i和作业j不满足Johnson不等式时, 交换它们的加工顺序后, 不增加加工时间。对于流水作业调度问题, 必存在最优调度 π , 使得作业 $\pi(i)$ 和 $\pi(i+1)$ 满足Johnson不等式。进一步还可以证明, 调度满足Johnson法则当且仅当对任意 $i < j$ 有

$$\min\{b_{\pi(i)}, a_{\pi(j)}\} \geq \min\{b_{\pi(j)}, a_{\pi(i)}\}$$

由此可知, 所有满足Johnson法则的调度均为最优调度。

算法描述

流水作业调度问题的Johnson算法

- (1) 令 $N_1 = \{i \mid a_i < b_i\}$, $N_2 = \{i \mid a_i \geq b_i\}$;
- (2) 将 N_1 中作业依 a_i 的非减序排序; 将 N_2 中作业依 b_i 的非增序排序;
- (3) N_1 中作业接 N_2 中作业构成满足Johnson法则的最优调度。

算法复杂度分析:

算法的主要计算时间花在对作业集的排序。因此, 在最坏情况下算法所需的计算时间为 $O(n \log n)$ 。所需的空间为 $O(n)$ 。

0-1背包问题

给定 n 种物品和一背包。物品 i 的重量是 w_i ，其价值为 v_i ，背包的容量为 C 。问应如何选择装入背包的物品，使得装入背包中物品的总价值最大？

0-1背包问题是一个特殊的整数规划问题。

$$\begin{aligned} & \max \sum_{i=1}^n v_i x_i \\ & \begin{cases} \sum_{i=1}^n w_i x_i \leq C \\ x_i \in \{0,1\}, 1 \leq i \leq n \end{cases} \end{aligned}$$

0-1背包问题

设所给0-1背包问题的子问题

$$\begin{cases} \max \sum_{k=i}^n v_k x_k \\ \sum_{k=i}^n w_k x_k \leq j \\ x_k \in \{0,1\}, i \leq k \leq n \end{cases}$$

的最优值为 $m(i, j)$ ，即 $m(i, j)$ 是背包容量为 j ，可选择物品为 $i, i+1, \dots, n$ 时0-1背包问题的最优值。由0-1背包问题的最优子结构性质，可以建立计算 $m(i, j)$ 的递归式如下。

$$m(i, j) = \begin{cases} \max\{m(i+1, j), m(i+1, j-w_i) + v_i\} & j \geq w_i \\ m(i+1, j) & 0 \leq j < w_i \end{cases}$$

$$m(n, j) = \begin{cases} v_n & j \geq w_n \\ 0 & 0 \leq j < w_n \end{cases}$$

0-1背包问题

```
template<class Type>
void knapsack(Type v, int w[], int c, int n, Type **m)
{
    int jMax = min( w[n]-1, c );
    for (int j=0; j <= jMax; j++)
        m[n][j] = 0;
    for (int j=w[n]; j <= c; j++)
        m[n][j] = v[n];
}
```

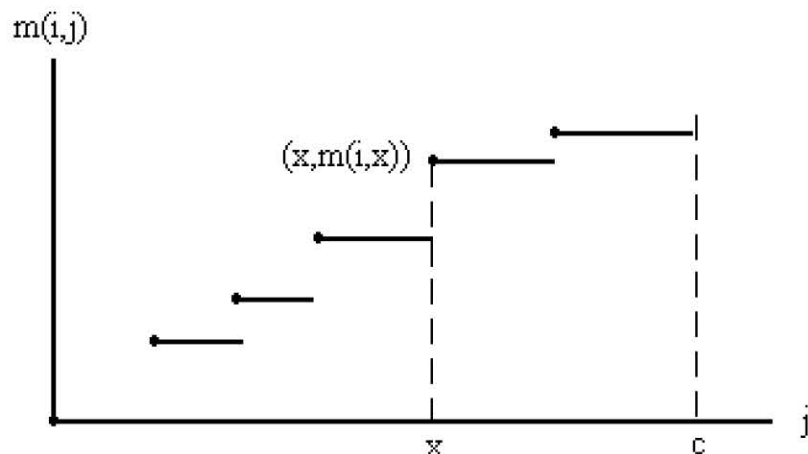
算法复杂度分析：

从 $m(i, j)$ 的递归式容易看出，算法需要 $O(nc)$ 计算时间。当背包容量 c 很大时，算法需要的计算时间较多。例如，当 $c > 2^n$ 时，算法需要 $\Omega(n2^n)$ 计算时间。

```
    m[i][j] = m[i+1][j];
    for( int j = w[i]; j <= c; j++)
        m[i][j] = max( m[i+1][j], m[i+1][ j-w[i] ] + v[i] );
}
m[1][c] = m[2][c];
if( c >= w[1] )
    m[1][c] = max( m[1][c], m[2][c-w[1]] + v[1] );
}
```

算法改进

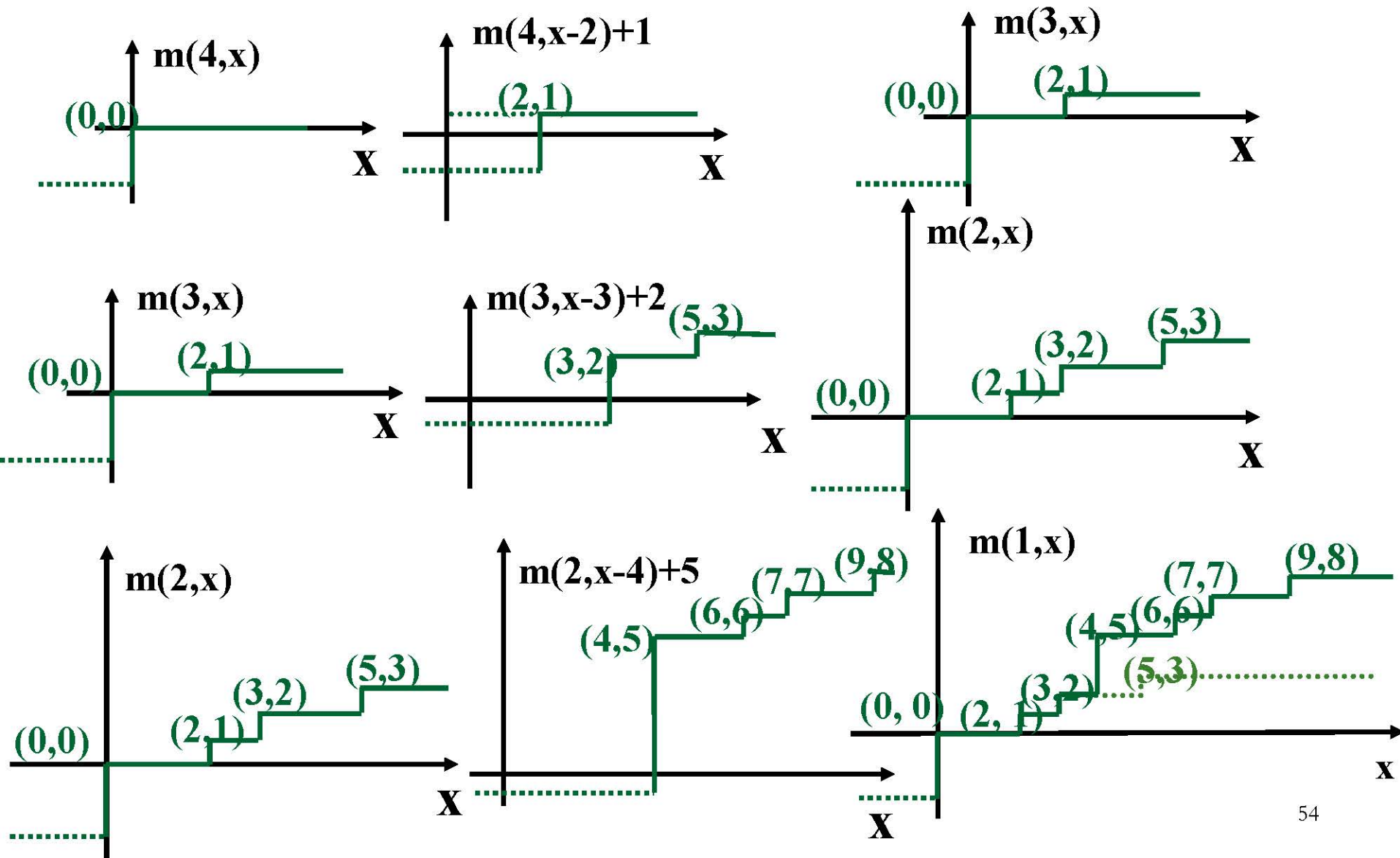
由 $m(i,j)$ 的递归式容易证明，在一般情况下，对每一个确定的 $i(1 \leq i \leq n)$ ，函数 $m(i,j)$ 是关于变量 j 的阶梯状单调不减函数。跳跃点是这一类函数的描述特征。在一般情况下，函数 $m(i,j)$ 由其全部跳跃点唯一确定。如图所示。



对每一个确定的 $i(1 \leq i \leq n)$ ，用一个表 $p[i]$ 存储函数 $m(i, j)$ 的全部跳跃点。表 $p[i]$ 可依计算 $m(i, j)$ 的递归式递归地由表 $p[i+1]$ 计算，初始时 $p[n+1]=\{(0, 0)\}$ 。

一个例子

$n=3$, $c=6$, $w=\{4, 3, 2\}$, $v=\{5, 2, 1\}$ 。



算法改进

- 函数 $m(i, j)$ 是由函数 $m(i+1, j)$ 与函数 $m(i+1, j-w_i)+v_i$ 作 $\max$ 运算得到的。因此，函数 $m(i, j)$ 的全部跳跃点包含于函数 $m(i+1, j)$ 的跳跃点集 $p[i+1]$ 与函数 $m(i+1, j-w_i)+v_i$ 的跳跃点集 $q[i+1]$ 的并集中。易知， $(s, t) \in q[i+1]$ 当且仅当 $w_i \leq s \leq c$ 且 $(s-w_i, t-v_i) \in p[i+1]$ 。因此，容易由 $p[i+1]$ 确定跳跃点集 $q[i+1]$ 如下
$$q[i+1] = p[i+1] \oplus (w_i, v_i) = \{(j+w_i, m(i, j)+v_i) | (j, m(i, j)) \in p[i+1]\}$$
- 另一方面，设 (a, b) 和 (c, d) 是 $p[i+1] \cup q[i+1]$ 中的2个跳跃点，则当 $c \geq a$ 且 $d < b$ 时， (c, d) 受控于 (a, b) ，从而 (c, d) 不是 $p[i]$ 中的跳跃点。除受控跳跃点外， $p[i+1] \cup q[i+1]$ 中的其它跳跃点均为 $p[i]$ 中的跳跃点。
- 由此可见，在递归地由表 $p[i+1]$ 计算表 $p[i]$ 时，可先由 $p[i+1]$ 计算出 $q[i+1]$ ，然后合并表 $p[i+1]$ 和表 $q[i+1]$ ，并清除其中的受控跳跃点得到表 $p[i]$ 。

一个例子

$n=5, c=10, w=\{2, 2, 6, 5, 4\}, v=\{6, 3, 5, 4, 6\}$ 。

初始时 $p[6]=\{(0,0)\}$, $(w_5, v_5)=(4,6)$ 。因此,

$q[6]=p[6]\oplus(w_5, v_5)=\{(4,6)\}$ 。

$p[5]=\{(0,0), (4,6)\}$ 。

$q[5]=p[5]\oplus(w_4, v_4)=\{(5,4), (9,10)\}$ 。从跳跃点集 $p[5]$ 与 $q[5]$ 的并集 $p[5]\cup q[5]=\{(0,0), (4,6), (5,4), (9,10)\}$ 中看到跳跃点 $(5,4)$ 受控于跳跃点 $(4,6)$ 。将受控跳跃点 $(5,4)$ 清除后, 得到

$p[4]=\{(0,0), (4,6), (9,10)\}$

$q[4]=p[4]\oplus(6, 5)=\{(6, 5), (10, 11)\}$

$p[3]=\{(0, 0), (4, 6), (9, 10), (10, 11)\}$

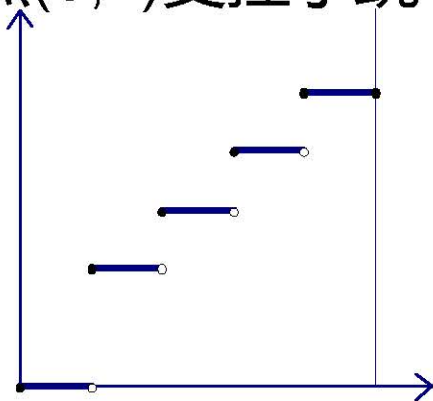
$q[3]=p[3]\oplus(2, 3)=\{(2, 3), (6, 9)\}$

$p[2]=\{(0, 0), (2, 3), (4, 6), (6, 9), (9, 10), (10, 11)\}$

$q[2]=p[2]\oplus(2, 6)=\{(2, 6), (4, 9), (6, 12), (8, 15)\}$

$p[1]=\{(0, 0), (2, 6), (4, 9), (6, 12), (8, 15)\}$

$p[1]$ 的最后的那个跳跃点 $(8,15)$ 给出所求的最优值为 $m(1,c)=15$ 。



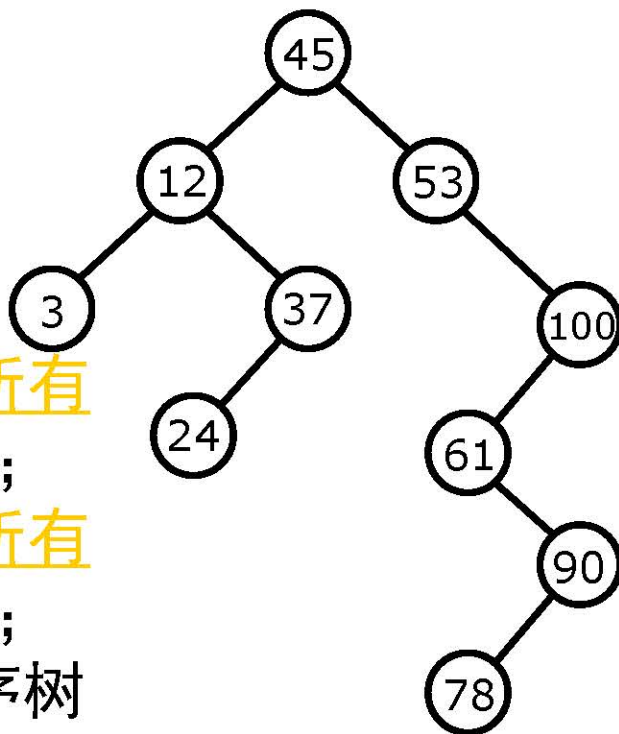
算法复杂度分析

上述算法的主要计算量在于计算跳跃点集 $p[i] (1 \leq i \leq n)$ 。由于 $q[i+1] = p[i+1] \oplus (w_i, v_i)$ ，故计算 $q[i+1]$ 需要 $O(|p[i+1]|)$ 计算时间。合并 $p[i+1]$ 和 $q[i+1]$ 并清除受控跳跃点也需要 $O(|p[i+1]|)$ 计算时间。从跳跃点集 $p[i]$ 的定义可以看出， $p[i]$ 中的跳跃点相应于 $x_i, \dots, x_n$ 的 0/1 赋值。因此， $p[i]$ 中跳跃点个数不超过 2^{n-i+1} 。由此可见，算法计算跳跃点集 $p[i]$ 所花费的计算时间为 $O\left(\sum_{i=2}^n |p[i+1]|\right) = O\left(\sum_{i=2}^n 2^{n-i}\right) = O(2^n)$ 。从而，改进后算法的计算时间复杂度为 $O(2^n)$ 。当所给物品的重量 $w_i (1 \leq i \leq n)$ 是整数时， $|p[i]| \leq c+1$ ， $(1 \leq i \leq n)$ 。在这种情况下，改进后算法的计算时间复杂度为 $O(\min\{nc, 2^n\})$ 。

最优二叉搜索树

■ 二叉搜索树

- (1) 若它的左子树不空，则左子树上所有节点的值均小于它的根节点的值；
- (2) 若它的右子树不空，则右子树上所有节点的值均大于它的根节点的值；
- (3) 它的左、右子树也分别为二叉排序树



在随机的情况下，二叉查找树的平均查找长度和 $\log n$ 是等数量级的

二叉搜索树的期望耗费

- 搜索成功与不成功的概率

$$\sum_{i=1}^n p_i + \sum_{i=0}^n q_i = 1$$

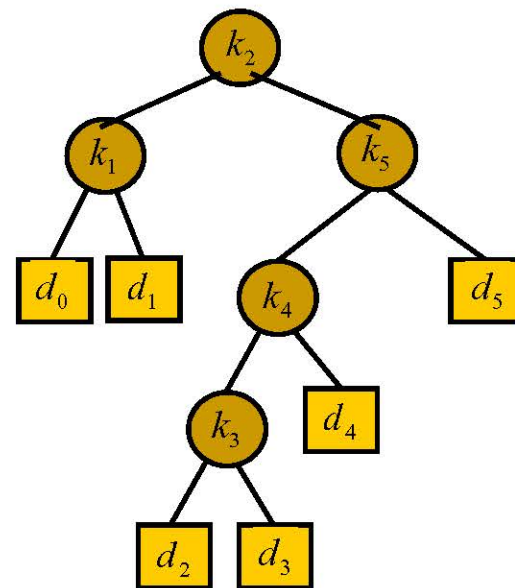
- 二搜索树的期望耗费

$E(\text{search cost in } T)$

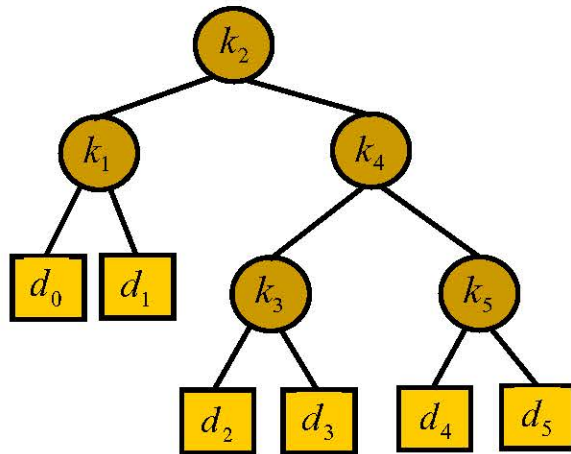
$$= \sum_{i=1}^n (\text{depth}_T(k_i) + 1) \cdot p_i + \sum_{i=0}^n (\text{depth}_T(d_i) + 1) \cdot q_i$$

$$= 1 + \sum_{i=1}^n \text{depth}_T(k_i) \cdot p_i + \sum_{i=0}^n \text{depth}_T(d_i) \cdot q_i$$

- 有 n 个节点的二叉树的个数为：
- 穷举搜索法的时间复杂度为指数级 $\Omega(4^n / n^{3/2})$



二叉搜索树的期望耗费示例



node	depth	probability	contribution
k_1	1	0.15	0.30
k_2	0	0.10	0.10
k_3	2	0.05	0.15
k_4	1	0.10	0.20
k_5	2	0.20	0.60
d_0	2	0.05	0.15
d_1	2	0.10	0.30
d_2	3	0.05	0.20
d_3	3	0.05	0.20
d_4	3	0.05	0.20
d_5	3	0.10	0.40
Total			2.80

最优二叉搜索树

最优二叉搜索树 T_{ij} 的平均路长为 p_{ij} ，则所求的最优值为 $p_{1,n}$ 。
由最优二叉搜索树问题的最优子结构性性质可建立计算 p_{ij} 的递归式如下

$$w_{i,j}p_{i,j} = w_{i,j} + \min_{i \leq k \leq j} \{w_{i,k-1}p_{i,k-1} + w_{k+1,j}p_{k+1,j}\}$$

记 $w_{i,j}p_{i,j}$ 为 $m(i,j)$ ，则 $m(1,n)=w_{1,n}p_{1,n}=p_{1,n}$ 为所求的最优值。计算 $m(i,j)$ 的递归式为

$$m(i,j) = w_{i,j} + \min_{i \leq k \leq j} \{m(i,k-1) + m(k+1,j)\}, \quad i \leq j$$

$$m(i,i-1) = 0, \quad 1 \leq i \leq n$$

注意到,

$$\min_{i \leq k \leq j} \{m(i,k-1) + m(k+1,j)\} = \min_{s[i][j-1] \leq k \leq s[i+1][j]} \{m(i,k-1) + m(k+1,j)\}$$

可以得到 $O(n^2)$ 的算法

第3章 小结

重点:

动态规划算法的子问题不独立，最优子结构性质，重叠子问题性质

难点:

动态规划解题过程:

- 找出最优解的性质，并刻画其结构特征。
- 递归地定义最优值。
- 以自底向上的方式计算出最优值。
- 根据计算最优值时得到的信息，构造最优解

分治法和动态规划

类型	分治法	动态规划
子问题结构	每个子问题不同	很多子问题重复
子问题数	多个子问题	多个子问题
子问题独立	独立	子问题相关
最优子结构	无要求	必须满足
问题与求解次序	自顶向下求解问题 (递归)	自底向上解子问题，再求更大规模问题

额外课程内容

算法设计与分析 北京大学 屈婉玲

<https://www.bilibili.com/video/av83623454/>

<https://www.coursera.org/learn/algorithms>

投资问题： m 元钱， n 项投资， $f_i(x)$: 将 x 元投入第 i 个项目的效益。求使得总效益最大的投资方案。

额外实践

<https://leetcode.cn/>